

Kaizen Classroom

TPI - 2026

Anthony Simond, SI-CA2a

Chef de projet : Raphael Favre

Expert 1 : Daniel Berney

Expert 2 : Jason Crisante



Kaizen
Classroom

Figure 1 - Logo

Date de Publication : jeudi 7 mai 2026

Table des matières

1	Analyse préliminaire	3
1.1	Introduction	3
1.2	Objectifs du projet	3
1.3	Planification initiale	5
2	Analyse / Conception.....	7
2.1	Technologies utilisées	7
2.2	Use Cases.....	8
2.3	Analyse des besoins	12
2.4	Analyse des risques	14
2.5	Planification	15
2.6	Maquettes	16
2.7	Modèle Conceptuel de Données	22
2.8	Choix technique du système de session	23
2.9	Chiffrements des mots de passes	24
2.10	Modèle Logique de Données.....	25
2.11	Stratégie de test	26
3	Réalisation.....	27
3.1	Architecture de l'application	27
3.2	Dossier de Réalisation	28

1 Analyse préliminaire

1.1 Introduction

Contexte et Objectifs :

Kaizen Classroom est une organisation à but non lucratif, sociale et solidaire, dont la finalité est de soutenir l'apprentissage des élèves en difficulté. Actuellement, le suivi des périodes de soutien (1 à 2 périodes par semaine) est géré à l'aide fichiers Excel ou googlesheet amenant à des interprétations diverses quant au remplissage par les enseignants ainsi qu'une forte complexité pour les administrateurs quant à la comptabilisation des périodes effectuées.

L'objectif de ce TPI est donc de réaliser une application qui simplifie la charge administrative des enseignants et des administrateurs de Kaizen Classroom

L'application doit permettre aux enseignants d'enregistrer rapidement les détails de chaque séance via une interface intuitive, d'automatiser des rapports destinés aux familles, tout en offrant aux administrateurs une visibilité rapide et synthétique des heures de suivi de chaque enseignant.

Ce projet est réalisé dans le cadre de mon Travail Pratique Individuel (TPI) de fin d'apprentissage au CPNV.

Ce projet me permet de consolider mes compétences en développement Python et en gestion de base de données MySQL, tout en utilisant un Framework moderne (Streamlit)

1.2 Objectifs du projet

1.2.1 Objectifs Fonctionnels (Rôles et fonctionnalités)

Le projet vise à fournir une interface différenciée selon deux rôles distincts :

Pour l'Enseignant :

- **Saisie du suivi hebdomadaires** : Formulaire permettant de noter la présence, le motif d'absence, le contenu pédagogique et les observations.
- **Consultation** : Accès à son propre historique avec un système de filtres par élève et par date.

Pour l'Administrateur :

- **Droits étendus** : Accès à toutes les fonctions « Enseignant ».
- **Supervision** : Vue globale sur les suivis de toute l'équipe avec des filtres par enseignant, par élève et par période.
- **Statistiques** : Calcul automatique du nombre de périodes effectuées par un enseignant sur un mois donné
- **Gestion (CRUD)** : L'administrateur aura accès à une interface lui permettant d'ajouter, modifier ou supprimer des entités (Élèves, Enseignants, Parents).
- **Attribution pédagogique** : L'administrateur aura accès à une interface permettant d'affecter un élève à un ou plusieurs enseignants spécifiques.

1.2.2 Objectifs Techniques et Contraintes

L'application doit respecter ces contraintes suivantes :

- **Développement** : Langage Python avec le Framework Streamlit.
- **Données** : Persistance via une base de données MySQL.
- **Sécurité** : Chiffrement des mots de passe en base de données et gestion stricte des sessions pour protéger l'accès administrateur.
- **Déploiement** : Accessibilité via navigateur web sur le réseau local de l'association.

1.2.3 Objectifs Personnels

- **Organisation** : Respect des délais des rendus (Tous les Mardi et Jeudi + Rendu Final)
- **Amélioration du Pré-TPI** : Finir les fonctionnalités du cahier des charges, Amélioration du Résumé du rapport du TPI dans les Annexes, Amélioration du Manuel d'Utilisation et aussi d'Installation, exécuter les tests que je n'ai pas pu faire dans le Pré-TPI.
- **Fonctionnalités** : Finir toutes les fonctionnalités prévues dans le cahier des charges.

1.3 Planification initiale

1.3.1 Méthodologie de Planification

Pour réaliser ce projet dans le temps imparti de 90 heures, j'ai choisi d'appliquer la méthodologie Waterfall (en cascade). Ce modèle linéaire est adapté aux projets dont les exigences sont clairement définies dès le départ, comme c'est le cas pour ce cahier des charges.

L'utilisation de cette approche se justifie par plusieurs facteurs :

- **Structure et rigueur** : Elle impose une validation de chaque étape avant de passer à la suivante, ce qui assure une base solide pour le développement.
- **Maîtrise des délais** : Avec un temps fixe de 90 heures, le découpage séquentiel permet de suivre précisément l'avancement et de respecter les échéances.
- **Expérience technique** : Ayant déjà mis en pratique cette méthodologie lors de mon Pré-TPI, je maîtrise ses outils et ses phases, ce qui réduit les risques d'erreurs d'organisation.

Conformément au cahier des charges, j'ai découpé le projet en quatre phases distinctes :

- Analyse/Conception (20h)
- Implémentation (34h)
- Tests (12h)
- Documentation(18h)
- Finalisation du TPI et Autres (6H)

1.3.2 Diagramme de Gantt

Mon planning prend en compte les contraintes horaires spécifiques (pauses, jours fériés des 14 et 15 mai) et respecte l'échéance finale du mercredi 20 mai 2026

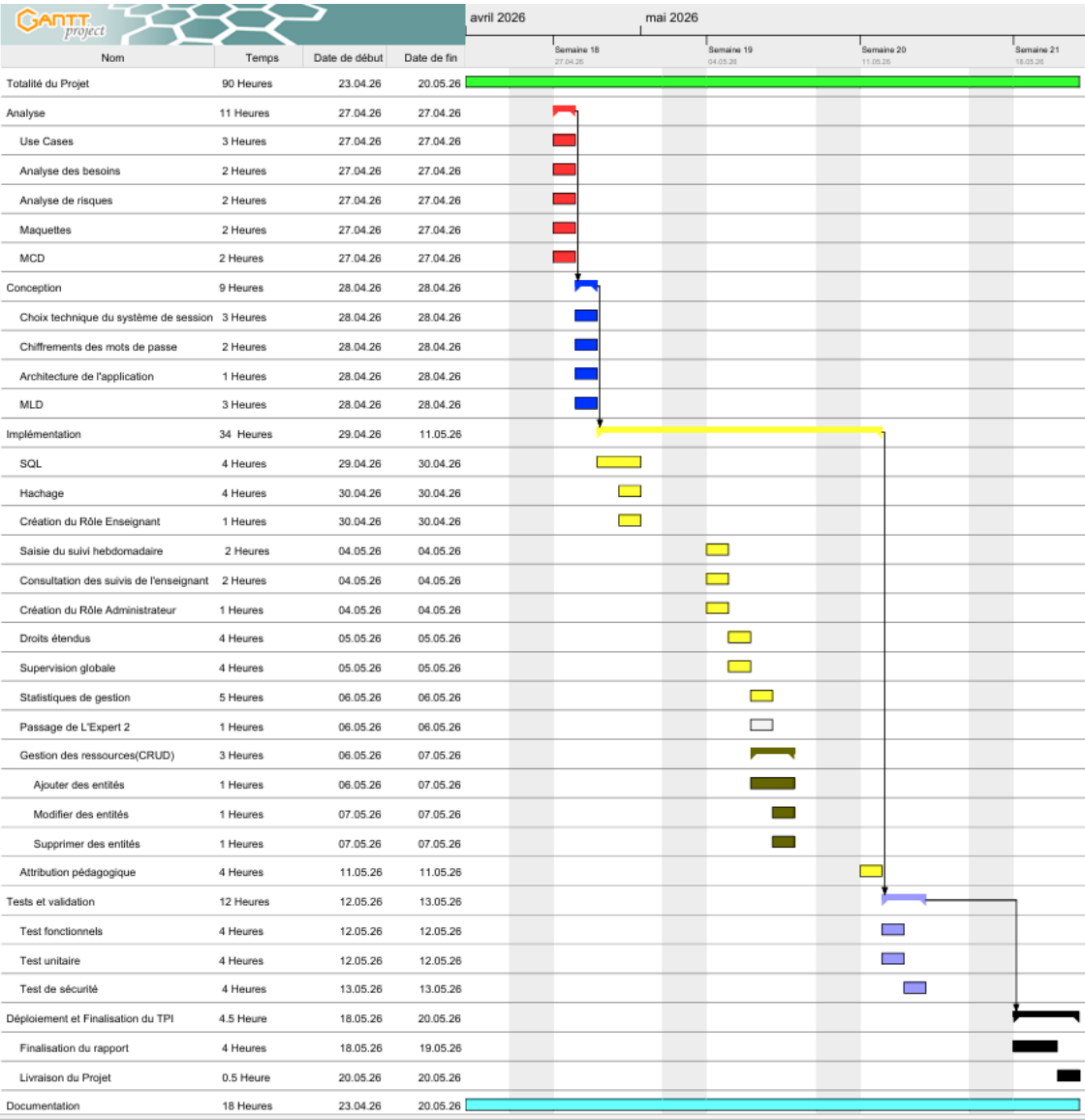


Figure 2 Planification Initiale

2 Analyse / Conception

2.1 Technologies utilisées

- **Python** : Langage principal du projet pour sa flexibilité et ses nombreuses bibliothèques orientées données.
- **PyCharm** : Environnement de développement utilisé pour l'écriture et le débogage du code.
- **Streamlit** : Framework utilisé pour l'interface utilisateur. Il permet de transformer des scripts Python en applications web interactives.
- **MySQL Workbench** : Outil de modélisation utilisé pour le MLD et l'administration de la base de données.
- **Serveur MySQL** : Système de gestion de base de données relationnelle qui assure le stockage, la persistance et la gestion des données du projet.
- **Git** : Logiciel de gestion de versions utilisé pour suivre l'évolution du code source. Il me permet de sauvegarder l'historique de mon travail, de revenir en arrière en cas d'erreur et de garantir l'intégrité de mon développement.
- **Gantt Project** : Utilisé pour établir la planification initiale.
- **Bcrypt** : Bibliothèque Python utilisé pour le hachage sécurisé des mots de passe. Elle repose sur l'algorithme de hachage Blowfish.
- **Draw.io** : Utilisé pour le MCD.
- **Figma** : Utilisé pour concevoir les interfaces visuelles durant la phase de « Maquettes »

2.1.1 Choix du SGBD : MySQL vs SQLite

Pour ce projet, le choix s'est porté sur MySQL plutôt que SQLite pour plusieurs raisons stratégiques :

- **Gestion Multi-utilisateurs (Accès concurrents) :**
 - SQLite est un simple fichier local. Si plusieurs enseignants essaient d'enregistrer un suivi en même temps, le fichier peut se verrouiller.

- MySQL est un serveur qui gère nativement de nombreux utilisateurs connectés simultanément, ce qui est indispensable pour une application de gestion de classe.
- **Sécurité et Droits d'accès :**
 - MySQL permet de créer des utilisateurs avec des privilèges précis (lecture seul, écriture, etc.) directement au niveau du serveur de données.
 - SQLite n'offre pas de gestion fine de droits d'accès au sein de la base.
- **Évolutivité :**
 - MySQL est conçu pour supporter de très gros volumes de données et peut être hébergé sur un serveur distant, permettant ainsi d'accéder à l'application depuis n'importe quel ordinateur de l'école.
- **Type de données et Fonctions :**
 - MySQL offre des types de données plus riches et des fonctions de calcul de temps plus puissantes que SQLite.

2.2 Use Cases

Acteurs identifiés :

1. **Enseignant** : L'utilisateur standard qui saisit les suivis.
2. **Administrateur** : l'utilisateur avec les pleins pouvoirs incluant aussi les droits de l'enseignant.

Tableau des cas d'utilisation :

ID	Acteur	Cas d'utilisation	Description
UC01	Utilisateur	S'authentifier	Se connecter de façon sécurisée (ID / Password).

ID	Acteur	Cas d'utilisation	Description
UC02	Enseignant	Saisir un suivi	Enregistrer présence, contenu et observations.
UC03	Enseignant	Consulter ses suivis	Voir son propre historique avec filtres (élève/date).
UC04	Admin	Superviser les suivis	Consulter les suivis de tous les enseignants.
UC05	Admin	Gérer les ressources (CRUD)	Ajouter/Modifier/Supprimer Enseignants, Élèves, Parents.
UC06	Admin	Affecter un élève	Lier un élève à un ou plusieurs enseignants.
UC07	Admin	Statistiques de Gestion	Calculer les périodes effectuées pour la facturation.

UC01 : S'authentifier

- **Acteur principal** : Utilisateur (Enseignant ou Administrateur).
- **Description** : Permet d'accéder à l'interface privée de l'application selon son rôle.
- **Précondition** : L'utilisateur possède un compte créé par un administrateur.
- **Scénario nominal** :
 1. L'utilisateur saisit son identifiant et son mot de passe.
 2. Le système vérifie la correspondance avec le hash stocké en base de données.
 3. Le système identifie le rôle (Enseignant/Admin) et redirige vers la page d'accueil correspondante.
- **Exceptions** : Si les identifiants sont erronés, le système affiche un message d'erreur et bloque l'accès.

UC02 : Saisir un suivi hebdomadaire

- **Acteur principal** : Enseignant (ou Administrateur)
- **Précondition** : L'enseignant doit être connecté.
- **Scénario nominal** :
 1. L'enseignant sélectionne l'élève (parmi ceux qui lui sont affectés).
 2. Il indique si l'élève est présent ou absent (avec motif).
 3. Il saisit le contenu pédagogique.
 4. Il ajoute des observations qualitatives.
 5. Il valide la saisie.
- **Postcondition** : Les données sont enregistrées dans la base MySQL.

UC03 : Consulter ses suivis

- **Acteur principal** : Enseignant.
- **Description** : Visualiser l'historique des séances passées pour préparer les suivantes.
- **Scénario nominal** :
 1. L'enseignant accède à l'onglet "Historique".
 2. Le système affiche uniquement les suivis rédigés par lui-même.
 3. L'enseignant utilise les filtres (par élève ou par plage de dates) pour affiner la liste.
 4. Le système met à jour l'affichage en temps réel.

UC04 : Supervision globale des suivis

- **Acteur principal** : Administrateur.
- **Description** : Contrôler le travail de l'ensemble du corps enseignant.
- **Scénario nominal** :
 1. L'administrateur accède à la vue "Supervision".
 2. Le système affiche les suivis de tous les enseignants.
 3. L'admin peut filtrer par enseignant spécifique, par élève ou par plage de dates pour vérifier la qualité des rapports.

UC05 : Gestion des ressources (CRUD)

- **Acteur principal** : Administrateur.
- **Description** : Gérer les entités de base (Enseignants, Élèves, Parents).
- **Scénario nominal** :
 1. L'admin sélectionne l'entité à gérer (ex: "Élèves").
 2. Il choisit l'action : **Ajouter** (nouveau formulaire), **Modifier** (éditer l'existant) ou **Supprimer**.
 3. Le système met à jour la base de données MySQL.
- **Contrainte** : La suppression d'un enseignant ou d'un élève doit être gérée prudemment (suppression en cascade) pour ne pas corrompre l'historique des suivis.

UC06 : Affectation pédagogique

- **Acteur principal** : Administrateur.
- **Description** : Lier un élève à un ou plusieurs enseignants.
- **Scénario nominal** :
 1. L'admin accède à l'interface d'affectation.
 2. Il sélectionne un enseignant dans une liste déroulante.
 3. Il sélectionne un ou plusieurs élèves à lui attribuer.
 4. Il valide l'affectation.
- **Postcondition** : Les élèves sélectionnés apparaîtront désormais dans la liste de saisie (UC02) de cet enseignant.

UC07 : Statistiques de gestion

- **Acteur principal** : Administrateur.
- **Description** : Permet de compter les heures pour la facturation.
- **Scénario nominal** :
 1. L'admin accède à la section "Statistiques".
 2. Il sélectionne un enseignant et une période (mois/année).

3. Le système interroge la base de données.
4. Le système affiche le total des périodes effectuées.

2.3 Analyse des besoins

Besoins fonctionnels :

Les besoins fonctionnels décrivent les actions que le système doit permettre de réaliser.

ID	Besoin	Description	Priorité
BF01	Authentification	Le système doit permettre aux utilisateurs de se connecter via un login et mot de passe.	Haute
BF02	Saisie de suivi	L'enseignant doit pouvoir saisir les données d'une séance (présence, contenu, observations).	Haute
BF03	Gestion des rôles	Le système doit limiter l'accès aux fonctions admin aux seuls utilisateurs autorisés.	Haute
BF04	Gestion CRUD	L'admin doit pouvoir créer, lire, modifier et supprimer les élèves, enseignants et parents.	Haute
BF05	Liaison Élève/Enseignant	L'admin doit pouvoir définir quel enseignant suit quel élève.	Haute
BF06	Calcul de périodes	Le système doit calculer automatiquement le total d'heures par enseignant/mois.	Moyenne
BF07	Historique filtrable	L'utilisateur doit pouvoir filtrer ses rapports par élève et par date.	Moyenne

Besoins non-fonctionnels :

Public cible :

Les utilisateurs finaux sont les enseignants de l'association, dont l'aisance technique peut varier. L'interface doit donc être très simple, épurée et efficace pour ne pas alourdir leur charge administrative. C'est pour cette raison que l'accent sera mis sur l'ergonomie.

Ces besoins définissent les contraintes de qualité, de sécurité et techniques.

Sécurité & Confidentialité

- **Protection des données** : Les mots de passe doivent être hachés avant d'être stockés en base de données.
- **Gestion des sessions** : Un utilisateur ne doit pas pouvoir accéder aux pages d'administration en manipulant l'URL.
- **Confidentialité** : Un enseignant ne peut voir que les données des élèves qui lui sont attribués.

Ergonomie & Utilisabilité

- **Simplicité** : L'interface Streamlit doit être épurée (public cible ayant une aisance technique variable).
- **Accessibilité** : L'application doit être consultable via n'importe quel navigateur web moderne sur le réseau local.

Performance & Fiabilité

- **Disponibilité** : L'application doit fonctionner de manière stable sur le serveur local de l'association.
- **Persistance** : Aucune donnée saisie ne doit être perdue en cas de redémarrage du serveur (utilisation de MySQL).

Maintenance

- **Code source** : Le code doit être versionné sur GitHub et être documenté pour permettre une reprise facile par un autre développeur.

2.4 Analyse des risques

Pour chaque risque, j'ai évalué la Probabilité (P) et la Gravité (G) sur une échelle de 1 à 3 (1 = faible, 3 = élevé). Le produit des deux donne l'Impact (I).

Risque	P	G	I	Mesure d'atténuation (Solution)
Perte de données (crash disque, mauvaise manipulation SQL)	1	3	3	Mettre en place des sauvegardes régulières de la base MySQL et utiliser Git pour le code.
Retard sur le planning (difficulté technique avec Streamlit)	2	2	4	Prioriser les besoins "Haute Priorité" et simplifier l'interface si le temps manque.
Accès non autorisé (faille de sécurité)	1	3	3	Hachage des mots de passe (bcrypt) et contrôle strict des sessions sur chaque page.
Incompatibilité réseau (accès local au CPNV/Association)	2	2	4	Tester l'accès via l'adresse IP locale très tôt dans la phase d'implémentation.
Problème matériel (ordinateur en panne)	1	3	3	Utilisation de GitHub (dépôt distant) pour récupérer le code rapidement sur une autre machine.
Complexité du filtrage SQL (statistiques complexes)	2	1	2	Se documenter sur les requêtes GROUP BY et COUNT en SQL dès la phase de conception.

2.5 Planification

Dans le cadre d'une méthodologie Waterfall, la planification a été figée dès le premier jour pour garantir le respect de l'échéance du 20 mai 2026.

Dates clés du projet :

Début du projet : Jeudi 23 avril 2026.

Fin du projet : Mercredi 20 mai 2026

Durée totale : 90 heures

Découpage des phases (WaterFall) :

Voici l'enchaînement des phases telles que définies dans mon diagramme de Gantt :

Phase d'Analyse et Conception (20h) :

- Définition des besoins
- Analyse des risques
- Use Cases
- MCD
- Maquettes
- Choix technique du système de session
- Chiffrements des mots de passe
- Architecture de l'application
- MLD

Phase de Réalisation / Implémentation (34h) :

- Création de la base de données
- Développement du code Python
- Interface Streamlit.

Phase de Tests (12h) :

- Tests unitaires
- Tests fonctionnels
- Test de sécurité
- Correction de bugs

Phase de Documentation (18h) :

- Rapport du TPI
- Manuel utilisateur

2.6 Maquettes

Pour créer les maquettes, j'ai utilisé l'IA dans Figma qui m'a permis de générer le design pour les maquettes, je lui ai dit ce que je voulais comme design et il me l'a généré.

Voici les différentes maquettes :

Page de Login :

Première page qui s'affiche quand on lance l'application.

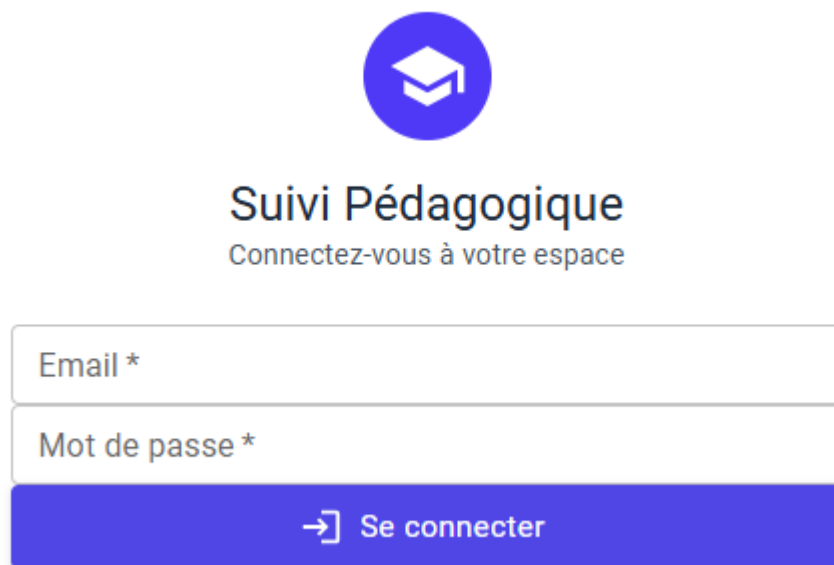
La maquette de la page de login est centrée sur un fond blanc. En haut, il y a un logo circulaire orange contenant un pictogramme d'un livre ouvert. Juste en dessous, le titre "Suivi Pédagogique" est écrit en une police sans-serif grasse, suivie du sous-titre "Connectez-vous à votre espace" en une police plus petite et grise. Ensuite, il y a deux champs de saisie rectangulaires à bordures grises : le premier est étiqueté "Email *" et le second "Mot de passe *". En bas, un bouton rectangulaire orange avec un effet de survol (une ombre portée) contient un pictogramme d'une flèche vers la droite à l'intérieur d'un carré, suivi du texte "Se connecter".

Figure 3 - Page Login

Page Enseignant - Saisie du Suivi hebdomadaire :

Quand on se connecte en tant qu'enseignant, c'est la première page qui s'affiche où l'on peut saisir le suivi de l'élève.

Espace Enseignant - M. Dupont DÉCONNEXION

[SAISIR UN SUIVI](#) [MES SUIVIS](#)

Saisie du suivi hebdomadaire

Élève
Date de la séance *
27.04.2026

Présence de l'élève

☒ Présent
☐ Absent

Contenu pédagogique abordé *

Observations et remarques

Enregistrer le suivi

Figure 4 - Page Enseignant Saisie du Suivi

Page Enseignant - Saisie du Suivi hebdomadaire avec Motif de l'absence :

Même page que celle du dessus sauf ici il y a le Motif de l'absence quand on clique sur le Bouton « Absent ».

Espace Enseignant - M. Dupont DÉCONNEXION

[SAISIR UN SUIVI](#) [MES SUIVIS](#)

Saisie du suivi hebdomadaire

Élève
Date de la séance *
04.05.2026

Présence de l'élève

☐ Présent
☒ Absent

Motif de l'absence

Contenu pédagogique abordé *

Observations et remarques

Enregistrer le suivi

Figure 5 - Page Enseignant Saisie du Suivi avec Motif d'absence

Page Enseignant - Consultation de ses suivis :

On peut filtrer par élève et par date.

Espace Enseignant - M. Dupont

SAISIR UN SUIVI

MES SUIVIS

Mes suivis

Filtrer par élève

Filtrer par date
jj.mm.aaaa

Date	Élève	Présence	Contenu	Observations	Actions
21/04/2026	Martin Dubois	Présent	Fractions et opérations	Très attentif, bonne progression	
20/04/2026	Sophie Leroy	Présent	Conjugaison - Passé composé	Quelques difficultés sur les auxiliaires	
19/04/2026	Lucas Bernard	Absent	-	Maladie (certificat médical)	
18/04/2026	Emma Petit	Présent	Géométrie - Les angles	Excellente participation	
14/04/2026	Martin Dubois	Présent	Tables de multiplication	Doit s'entraîner davantage	

Total: 5 suivi(s)

Figure 6 - Consultation des Suivis

Page Admin - Statistiques de gestion :

Espace Administrateur - Admin Principal

STATISTIQUES

SAISIR UN SUIVI

TOUS LES SUIVIS

GESTION CRUD

AFFECTATIONS

Statistiques et gestion

Suivis ce mois
142

Enseignants actifs
12

Taux de présence
94%

Élèves suivis
48

Décompte des périodes par enseignant

Mois
Avril 2026

Enseignant
Tous les enseignants

Enseignant	Nombre de séances	Élèves différents	Heures totales
M. Dupont	18	8	27h
Mme Martin	22	10	33h
M. Rousseau	15	6	22.5h
Mme Durand	20	9	30h
M. Lefevre	12	5	18h
Total	87	38	130.5h

Figure 7 - Statistiques

Page Admin - Tous les Suivis :

l'admin a accès à tous les suivis de l'ensemble du corps enseignant avec des filtres.

Espace Administrateur - Admin Principal DECONNEXION

STATISTIQUES SAISIR UN SUIVI **TOUS LES SUIVIS** GESTION CRUD AFFECTATIONS

Tous les suivis

Filtrer par élève Filtrer par enseignant Filtrer par date jj.mm.aaaa

Date	Élève	Enseignant	Présence	Contenu	Observations	Actions
21/04/2026	Martin Dubois	M. Dupont	Présent	Fractions et opérations	Très attentif, bonne progression	
20/04/2026	Sophie Leroy	M. Dupont	Présent	Conjugaison - Passé composé	Quelques difficultés sur les auxiliaires	
19/04/2026	Lucas Bernard	Mme Martin	Absent	-	Maladie (certificat médical)	
18/04/2026	Emma Petit	M. Dupont	Présent	Géométrie - Les angles	Excellente participation	
14/04/2026	Martin Dubois	M. Dupont	Présent	Tables de multiplication	Doit s'entraîner davantage	

Total: 5 suivi(s)

Figure 8 - Tous les Suivis

Page Admin - Gestion des ressources (CRUD) :

Ajouter, Modifier et Supprimer des entités (Elèves, Enseignants et Parents)

Espace Administrateur - Admin Principal DECONNEXION

STATISTIQUES SAISIR UN SUIVI TOUS LES SUIVIS **GESTION CRUD** AFFECTATIONS

Gestion des entités (CRUD)

ÉLÈVES **ENSEIGNANTS** PARENTS

+ Ajouter un élève

Nom	Classe	Parent	Email	Actions
Martin Dubois	CM2	M. et Mme Dubois	dubois@email.fr	
Sophie Leroy	CM1	Mme Leroy	leroy@email.fr	
Lucas Bernard	CE2	M. Bernard	bernard@email.fr	
Emma Petit	CM2	M. et Mme Petit	petit@email.fr	

Figure 9 - Gestion des entités (CRUD)

Pop-up - Modification élèves :**Modifier un élève**

Nom complet
Email
Classe
Date de naissance jj.mm.aaaa 

ANNULER

MODIFIER

*Figure 10 - Pop-up Modification d'élève***Pop-up - Ajout d'élève :****Ajouter un élève**

Nom complet
Email
Classe
Date de naissance jj.mm.aaaa 

ANNULER

AJOUTER

Figure 11 - Pop-up Ajout d'élève

Page Admin – Attribution pédagogique :

Espace Administrateur - Admin Principal

DÉCONNEXION

STATISTIQUES

SAISIR UN SUIVI

TOUS LES SUIVIS

GESTION CRUD

AFFECTATIONS

Attribution pédagogique

Sélectionner un enseignant

Sélectionner un élève

Attribuer l'élève à l'enseignant

M. Dupont

Mathématiques

Élèves affectés (3)

Martin Dubois

Emma Petit

Sophie Leroy

Mme Martin

Français

Élèves affectés (2)

Sophie Leroy

Lucas Bernard

M. Rousseau

Sciences

Élèves affectés (2)

Lucas Bernard

Emma Petit

Figure 12 - Attribution pédagogique

2.7 Modèle Conceptuel de Données

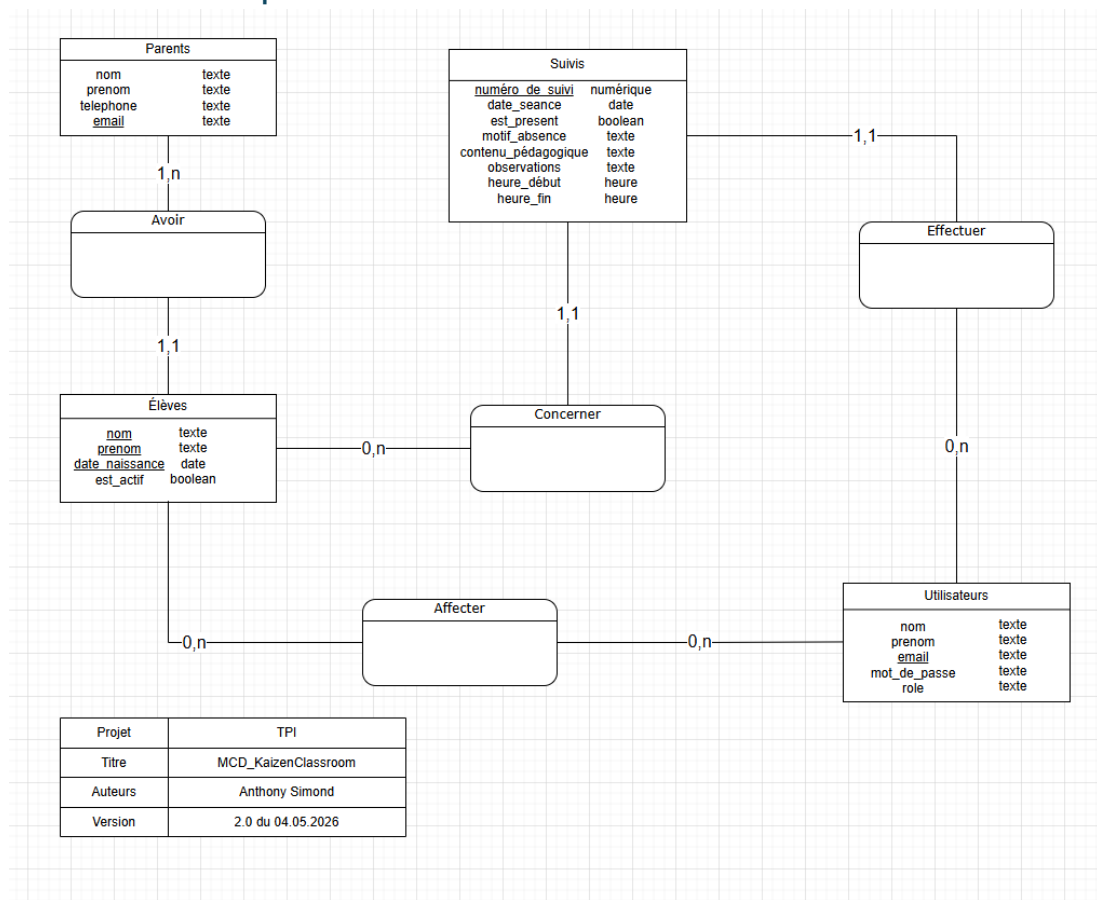


Figure 13 – MCD

1^{ère} version du MCD :

Il manquait les identifiants naturels souligné pour les tables Éèves et Suivis.
Mise en page était brouillon.

2^{ème} Version du MCD :

- Ajout des identifiants naturels pour la table Éèves et Suivis.
- Ajout du champ « numéro_de_suivi » dans la table Suivis.
- Ajout du champ « est_actif » dans la table Éèves.
- Ajout des champs « heure_debut et heure_fin »
- Changement de cardinalité de « 1,n à 0,n » pour relation entre Éèves à Utilisateurs

Spécificité des relations entre Utilisateurs et Suivis :

Un nouvel enseignant n'a pas encore rédigé de suivis à son arrivé, c'est pour cela que la cardinalité est de « 0, n » au lieu de « 1, n ».

Spécificité des relations entre Élèves et Suivis :

Un élève peut ne pas avoir encore de suivis, c'est pour cela que la cardinalité est de « 0, n » au lieu de « 1, n ».

Spécificité des relations entre Utilisateurs et Élèves (Relation Affecter) :**Du côté des Utilisateurs :**

Un utilisateur (enseignant ou administrateur) peut être enregistré dans le système sans avoir encore d'élèves attribués à sa charge. C'est pour cela que la cardinalité est de « 0, n » au lieu de « 1, n ». Cela permet de créer le compte d'un nouvel enseignant avant même qu'il ne commence ses séances.

Du côté des Élèves :

Un élève peut être inscrit dans la base de données mais ne pas encore être affecté à un enseignant spécifique. C'est pour cela que la cardinalité est de « 0, n » au lieu de « 1, n ».

2.8 Choix technique du système de session

Pour maintenir la connexion de l'utilisateur et gérer les droits d'accès (Enseignant ou Administrateur) tout au long de sa navigation, j'ai choisi d'utiliser le module natif de Streamlit : `st.session_state`.

Justification du choix

Le `session_state` est un dictionnaire Python persistant durant toute la durée de la session de l'utilisateur sur le navigateur. Il est idéal pour ce projet car :

- **Simplicité** : Il permet de stocker facilement des variables (ex : `logged_in = True`) sans avoir besoin de cookies complexes.
- **Sécurité côté serveur** : Contrairement aux cookies classiques stockés chez l'utilisateur, les données de `session_state` restent sur le serveur, ce qui limite les risques de manipulation des rôles par l'utilisateur.
- **Réactivité** : Streamlit recharge la page à chaque interaction ; le `session_state` assure que l'utilisateur ne soit pas déconnecté à chaque clic.

2.9 Chiffrements des mots de passes

Afin de garantir la confidentialité des comptes utilisateurs et de respecter les bonnes pratiques de sécurité, les mots de passe ne seront jamais stockés en « clair » (lisibles) dans la base de données MySQL.

Choix technique : la bibliothèque bcrypt

J'ai choisi d'utiliser la bibliothèque bcrypt pour gérer la sécurisation des accès. C'est un standard de l'industrie pour plusieurs raisons :

- **Salage automatique (Salt)** : bcrypt génère un « sel » aléatoire pour chaque mot de passe, ce qui empêche deux utilisateurs ayant le même mot de passe d'avoir le même hash en base de données. Cela protège contre les attaques « Rainbow Tables »
- **Lenteur adaptive** : Contrairement à MD5 ou SHA-1 (qui sont obsolètes), bcrypt est volontairement un peu lent, ce qui rend les attaques par "Brute Force" très difficiles.
- **Compatibilité Python** : Elle s'intègre parfaitement avec Streamlit et MySQL.

2.10 Modèle Logique de Données

Le MLD a été fait avec MySQL Workbench.

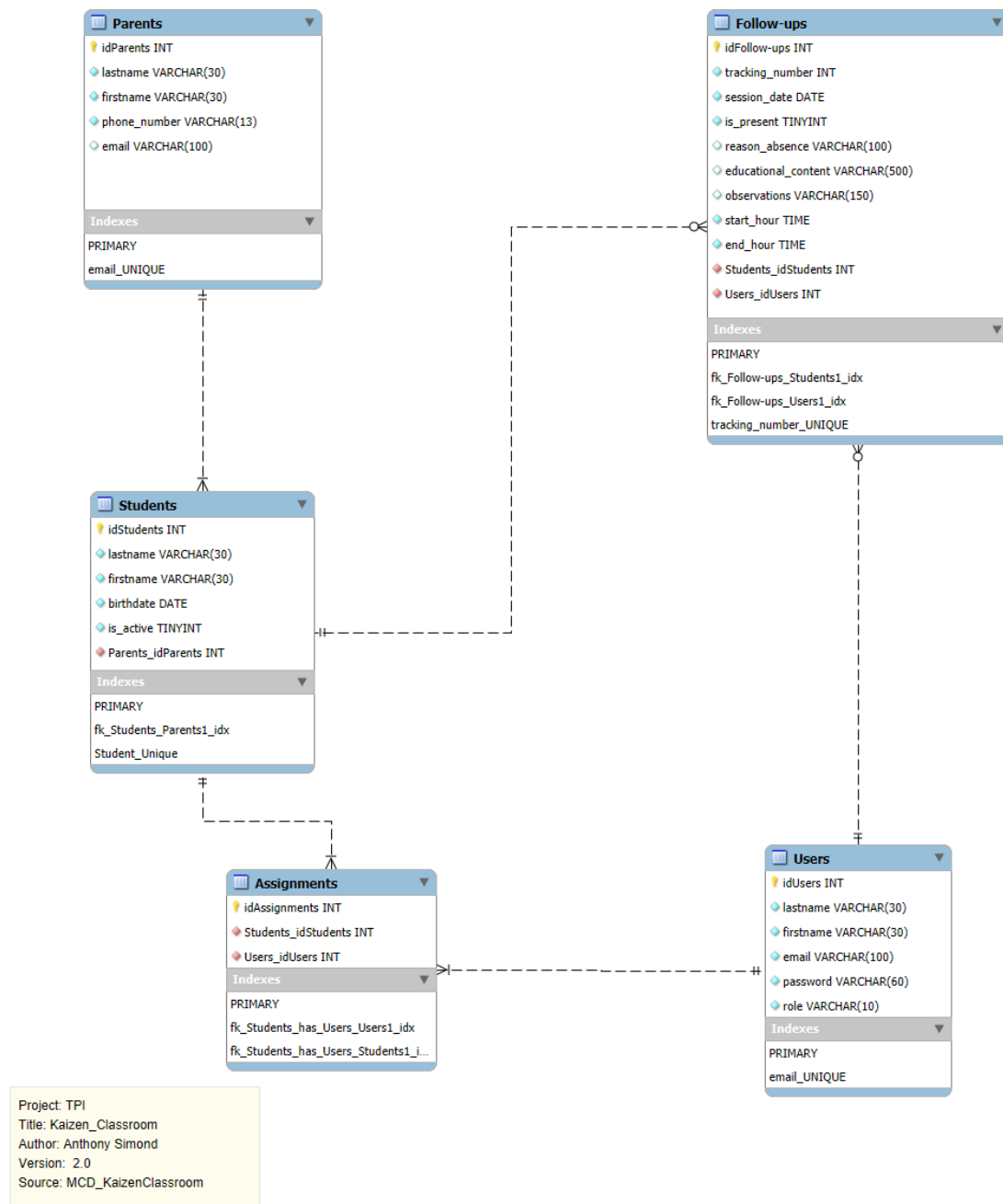


Figure 14 - MLD

La table « Assignments » est une relation de type « n, n » (Plusieurs-à-Plusieurs), elle devient donc une table à part entière. Elle contient les deux IDs pour lier un enseignant à un élève.

Types de données :

le mot de passe dans la table « Users » a une longueur de 60 caractères pour accueillir le hash généré par bcrypt.

Les emails sont marqués comme UNIQUE pour éviter les doublons de comptes.

2.11 Stratégie de test

Ma stratégie de test vise à garantir que l'application Kaizen Classroom est non seulement fonctionnelle, mais aussi sécurisée et fiable pour les enseignants et administrateurs.

Type de tests et ordre d'exécution :

1. **Tests Unitaires** : Validation individuelle des fonctions de sécurité (hachage et vérification des mots de passe dans security.py).
2. **Tests d'Intégration** : Vérification de la communication entre l'application Python et la base de données MySQL (connexion et exécution de requêtes).
3. **Tests Fonctionnels** : Vérification du flux complet : connexion utilisateur -> redirection selon le rôle -> affichage des données des élèves.
4. **Tests de Validation (Interface)** : Test des composants visuels de Streamlit

Moyens à mettre en œuvre :

- **Outils** : HeidiSQL pour vérifier l'état des tables après les manipulations
- **Automatisation** : Utilisation d'un script de Seeding (seed_users.py) pour réinitialiser un environnement de test propre et sécurisé en un clic.

Couverture des tests :

Les tests ne seront pas exhaustifs. Il est impossible de tester toutes les combinaisons de saisie dans le temps imparti.

- **Focus** : Je privilégie les "Chemins Critiques" (Authentification, accès aux données élèves, saisie des suivis).
- **Cas limites** : Je teste les erreurs courantes (mauvais mot de passe, email inexistant, champs vides) pour m'assurer que l'application ne "plante" pas mais affiche une erreur compréhensible.

3 Réalisation

3.1 Architecture de l'application

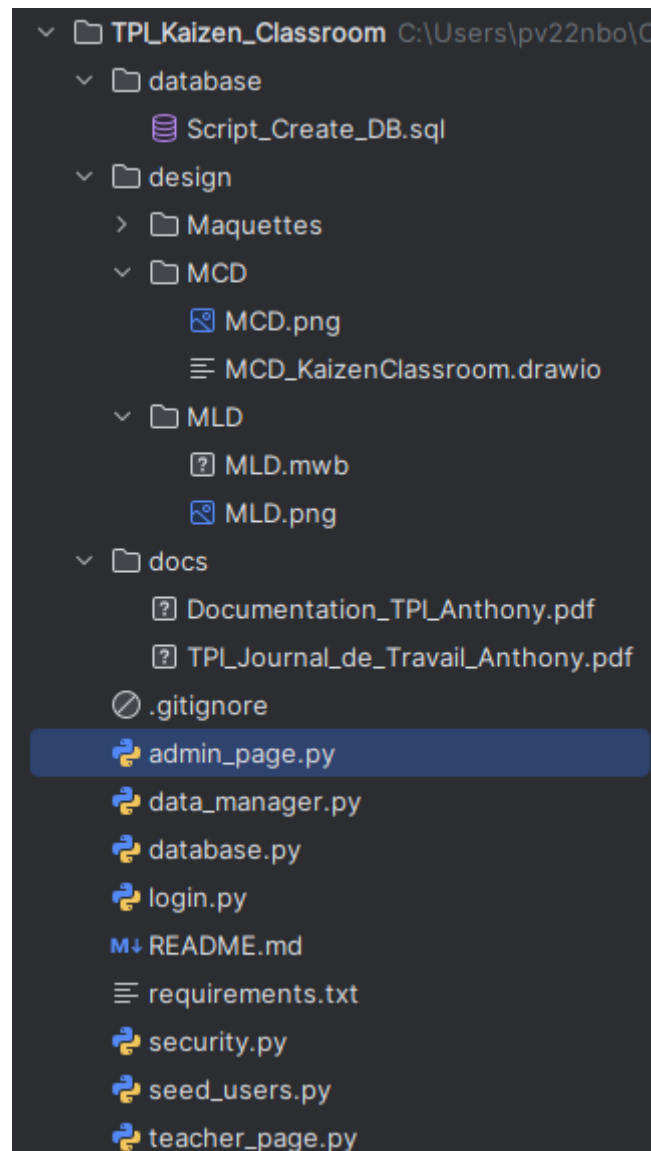


Figure 15 - Architecture de l'application

3.2 Dossier de Réalisation

3.2.1 Environnement logiciel et matériel :

- **Matériel** : Ordinateur du CPNV.
- **Système d'exploitation** : Windows 11
- **Outils de développement** : Pycharm 2025.2.0.1, MySQL Workbench pour le MLD, Draw.io pour le MCD.
- **Versions des langages** : Python 3.13 et MySQL 8.0.

3.2.2 Répertoire et fichiers :

Fichiers de Logique et de Données :

Ces fichiers gèrent l'interaction avec la base de données et la sécurité.

- `database.py` : Contient la configuration de base de la connexion MySQL et aussi la vérification du login. C'est ici que l'application établit le pont avec le serveur de base de données.
- `data_manager.py` : C'est le « moteur » de données. Il contient toutes les fonctions SQL qui extraient et formatent les données brutes pour les pages.
- `security.py` : Gère la protection des données avec le hachage des mots de passe.

Fichiers d'Interface Utilisateur :

Ces fichiers gèrent ce que l'utilisateur voit à l'écran grâce à Streamlit.

- `login.py` : Le point d'entrée de l'application. Il gère l'authentification et redirige l'utilisateur vers la bonne page selon son rôle (Admin ou Enseignant).
- `admin_page.py` : Contient toute l'interface destinée aux administrateurs.
- `teacher_page.py` : Interface dédiée aux enseignants pour la saisie des suivis et la consultation de leurs propres activités.

Dossiers de Conception et Ressources :

- /database : Contient Script_Create_DB.sql, le script permettant de recréer l'intégralité de la structure de la base de données.
- /design : Regroupe la partie réflexion du projet :
 - MCD et MLD : Schémas conceptuels et logiques de la base de données.
 - Maquettes : les dessins préparatoires de l'interface utilisateur.
- /docs : Journal de travail et documentation en PDF.

Fichiers de Configuration et Utilitaires :

- requirements.txt : Liste de toutes les bibliothèques Python nécessaires pour installer le projet sur une nouvelle machine.
- seed_users.py : Un script utilitaire pour remplir la base de données avec des utilisateurs de test lors du premier lancement.
- .gitignore : Indique à Git de ne pas sauvegarder les fichiers inutiles ou sensibles.
- README.md : Le guide d'accueil qui explique comment installer et lancer le projet.

3.2.3 Programmation et Dépendances :

Librairies externes :

- Streamlit
- Bcrypt
- Mysql-connector-python
- Python-dotenv

Reconstruction du Logiciel :

1. Prérequis système

- **Python 3.13+** : Le langage de programmation utilisé.
- **MySQL Server** : Pour la gestion de la base de données.
- **Git** : Pour la récupération des sources.

2. Récupération des sources et dépendances

Après avoir cloné le dépôt ou extrait l'archive, il est nécessaire d'installer les bibliothèques Python requises. L'installation est automatisée via le fichier requirements.txt :

```
pip install -r requirements.txt
```

3. Configuration de la base de données

1. **Création des tables** : Importer le script SQL fourni (/database/Script_Create_DB.sql) dans votre instance MySQL.
2. **Sécurisation des accès** :
 - Le logiciel utilise des variables d'environnement pour ne pas stocker d'identifiants en clair dans le code.
 - Renommer le fichier .env.example en .env.
 - Modifier les valeurs (DB_HOST, DB_USER, DB_PASSWORD) pour correspondre à votre configuration locale.

4. Amorçage des données (Seeding)

Pour pouvoir se connecter et tester l'application , un script d'initialisation doit être exécuté :

```
« python seed_users.py »
```

5. Lancement de l'application

L'interface utilisateur est lancée via le framework Streamlit :

```
« streamlit run login.py »
```

3.2.4 Codes :

La Sécurité : le hachage avec BCrypt :

```
def add_teacher(lastname,firstname,email,password_hash): 1 usage
    """
    Adds a teacher with a hashed password to the database.
    :param lastname: Lastname of the teacher.
    :param firstname: Firstname of the teacher.
    :param email: Email of the teacher.
    :param password_hash: Password hashed of the teacher.
    :return: True if successful, False otherwise.
    """
    try:
        password_bytes = password_hash.encode('utf-8')
        salt = bcrypt.gensalt()
        hashed_password = bcrypt.hashpw(password_bytes, salt)

        conn = get_connection()
        cursor = conn.cursor()
        query = """
        INSERT INTO users (lastname, firstname, email, password,role)
        VALUES (%s, %s, %s, %s, 'Enseignant')
        """
        cursor.execute(query, (lastname, firstname, email, hashed_password.decode('utf-8')))
        conn.commit()
        conn.close()
        return True
    except Exception as e :
        print(f"Error hachage/insertion:{e}")
        return False
```

Figure 16 - Fonctions Ajout Enseignant

Pour garantir la confidentialité des utilisateurs, j' ai implémenté la bibliothèque BCrypt. Avant l'insertion en base de données, le mot de passe est 'salé' et haché. Cela signifie que même en cas de vol de la base de données, les mots de passe originaux restent indéchiffrables.

La Logique SQL : Requêtes avec jointures et filtres dynamiques

```

query = """
SELECT f.session_date , s.firstname, s.lastname, f.is_present,
       f.educational_content, f.observations, f.reason_absence
FROM `follow-ups` f
JOIN students s ON f.Students_idStudents = s.idStudents
WHERE f.Users_idUsers = %s
"""

params = [teacher_id]

if student_id:
    query += " AND f.Students_idStudents = %s "
    params.append(student_id)

```

Figure 17 - Requête SQL

Cette fonction utilise une jointure SQL (JOIN) pour lier les rapports de suivi aux noms des élèves. J'ai mis en place une construction de requête dynamique : les filtres (par élève ou par date) ne sont ajoutés à la requête SQL que si l'utilisateur les sélectionne dans l'interface, optimisant ainsi les performances et la précision des résultats.

La Robustesse : Variables d'environnement

```

load_dotenv()

def get_connection(): 19 usages  ⚡ Anthony Simond
    """
    Creates and returns a connection to the MySQL database.
    """
    return mysql.connector.connect(
        host=os.getenv("DB_HOST"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME"),
    )

```

Figure 18 Fonctions Connexion BD

Afin de respecter les bonnes pratiques de développement, les identifiants de connexion ne sont pas écrits en dur dans le code mais chargés via un fichier .env .